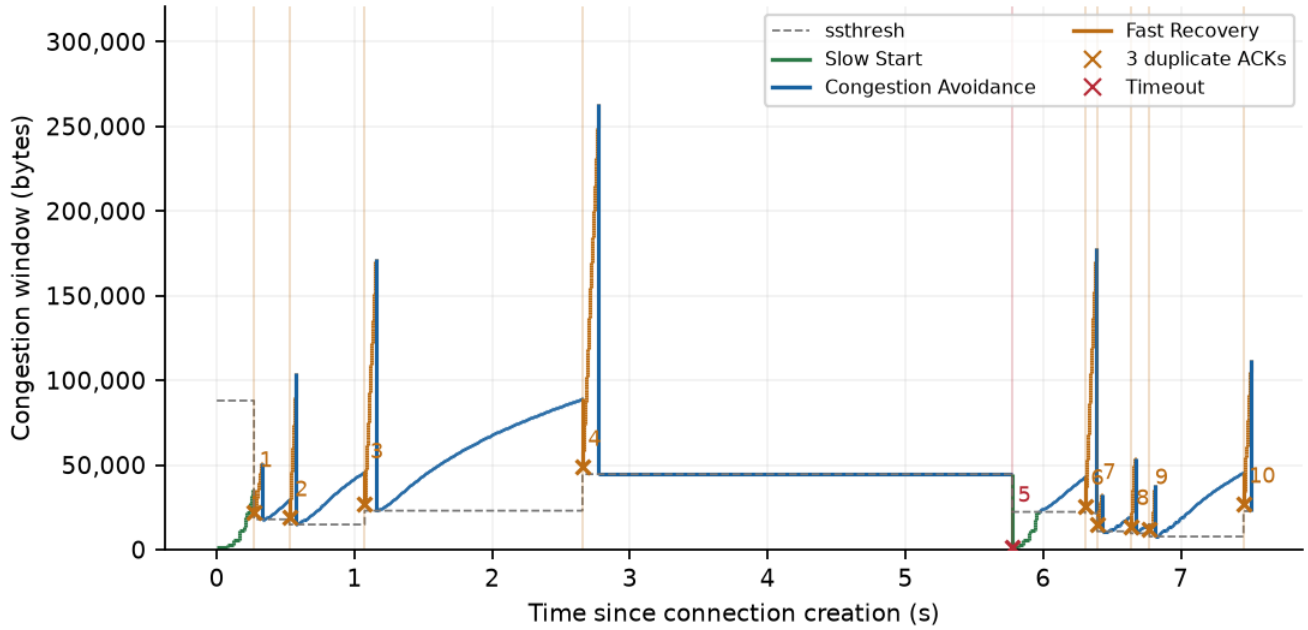
1. Congestion Control Report

15-441/641 - TCP in the Wild - Checkpoint 2

1.1 Sender's congestion window (bytes) over time

Selected connection: 0.2% sender loss, trial 8, 2,097,152 bytes. Transfer time: 7.512 s. The full trace contains 9 triple-duplicate-ACK events and 1 timeout. Colors identify the sender state after each event; numbered marks identify every loss response.



Mark	Time (s)	Event	cwnd after (B)	ssthresh (B)
1	0.270	3 duplicate ACKs	22,000.0	17,875.0
2	0.527	3 duplicate ACKs	18,926.3	14,801.3
3	1.074	3 duplicate ACKs	27,102.4	22,977.4
4	2.655	3 duplicate ACKs	48,683.3	44,558.3
5	5.776	Timeout	1,375.0	22,279.2
6	6.306	3 duplicate ACKs	25,730.6	21,605.6
7	6.392	3 duplicate ACKs	14,927.8	10,802.8
8	6.635	3 duplicate ACKs	13,757.5	9,632.5
9	6.772	3 duplicate ACKs	11,828.4	7,703.4
10	7.455	3 duplicate ACKs	26,835.1	22,710.1

Source trace: loss-0.2-run-08.csv

1. Congestion Control Report (continued)

1.2 Does the observed CWND behavior match TCP Reno?

Partly: the state transitions match, but the measured build overcredits short ACKs. These AWS measurements use source snapshot 7d45e18, before that correction. In **slow start** (green), each full-segment ACK adds one MSS, giving roughly exponential growth per RTT. In **congestion avoidance** (blue), the measured build adds MSS squared divided by cwnd per ACK, even for short segments. That can exceed Reno's one-MSS increase per flight. The corrected build uses MSS*min(N,MSS)/cwnd, where N is newly acknowledged bytes; whole-flight and split-ACK regressions verify roughly one MSS of growth. The plotted AWS data have not been rerun with this correction.

Fast retransmit and fast recovery (orange) begin after three duplicate ACKs: ssthresh becomes half the previous cwnd, the missing segment is retransmitted, and cwnd becomes ssthresh + 3*MSS. For example, at 0.270 s, cwnd was 35,750.0 bytes; ssthresh becomes 17,875.0 bytes and cwnd becomes 22,000.0 bytes. Each further duplicate ACK adds one MSS. The next advancing ACK exits recovery and deflates cwnd to ssthresh. This explains the tall recovery peaks and subsequent reductions. New transmissions are also limited by the advertised receive window.

Timeout recovery: at 5.776 s, cwnd falls from 44,558.3 to 1,375.0 bytes (one MSS), ssthresh becomes 22,279.2 bytes, and slow start resumes. The flat interval before this event reflects waiting for the fixed three-second RTO. Multiple losses within one Reno window can require a timeout after fast recovery; the trace need not form a perfectly regular sawtooth.

1.3 Experiment setup: bandwidth, loss, and transfer size

Setting	Value
Placement	Two Ubuntu 24.04 t3.micro instances, us-west-2a
Path / shaping	Private IPv4, same VPC; tcconfig on each node's ens5
Rate / delay	5 Mbps and 20 ms egress delay on each node (~40 ms RTT)
CWND plot scenario	0.2% sender loss, trial 8; receiver loss 0%
Mathis loss sweep	0.1%, 0.2%, 0.5%, 1%, 2%, 5%, 10%; sender only
Repetitions / file	10 at each rate; 2,097,152 bytes per transfer
Protocol parameters	MSS 1375 B; initial cwnd 1375 B; initial ssthresh 88,000 B
Buffer / timeout	65,535 bytes per direction; fixed RTO 3000 ms

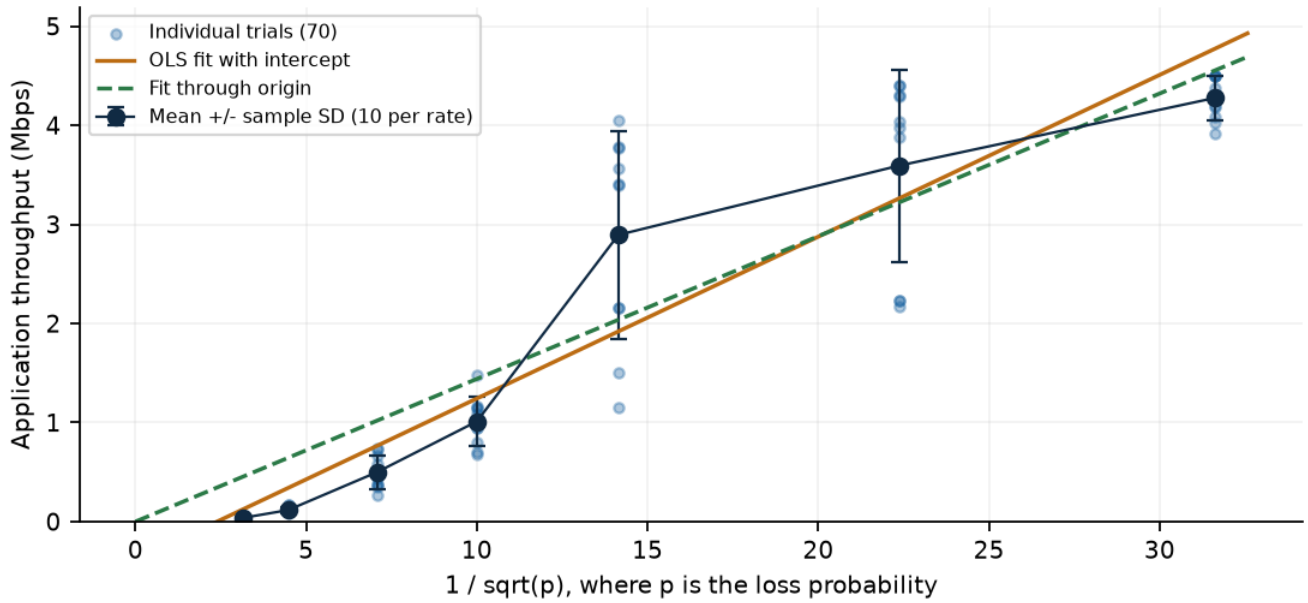
Bandwidth, delay, payload, and protocol settings remain fixed while sender loss varies. The same deterministic random 2 MiB file is used in all 70 transfers. Durations range from 3.728 to 487.274 seconds, so every transfer lasts at least a few seconds. Every received file passed SHA-256 verification. No trials failed.

Timing starts just after socket creation and ends at the application receipt confirming that the receiver wrote the entire file. It includes any remaining connection establishment and the final receipt, but excludes FIN and TIME_WAIT. Application throughput in Mbps is 8 * file bytes / (duration in seconds * 1,000,000); retransmissions and headers are excluded.

2. Mathis Model Evaluation

2.1 Throughput against 1/sqrt(p), with linear regression

Following writeup Section 3.2.2, the hypothesis is T = C/sqrt(p), where p is the configured loss probability expressed as a fraction (0.1% means p = 0.001). The figure plots all 70 measurements against x = 1/sqrt(p), connects the seven group means, and shows sample-standard-deviation error bars for ten trials per setting. Both regression lines use all 70 measurements.



Loss (%)	Mean (Mbps)	Sample SD	Mean duration (s)	Timeouts (total)
0.1	4.27626	0.22526	3.93	0
0.2	3.59226	0.97069	5.09	3
0.5	2.89308	1.04914	6.89	7
1	1.01111	0.24706	17.53	38
2	0.49839	0.16848	37.44	97
5	0.11646	0.02989	151.48	458
10	0.03853	0.00360	438.71	1394

Unconstrained OLS: T = 0.163281 x -0.390209 Mbps, with R squared = 0.83361. **Hypothesis-constrained regression:** T = 0.144013 x Mbps, with intercept fixed at zero. The latter directly estimates C for the equation in the writeup.

2. Mathis Model Evaluation (continued)

2.2 What value did you find for C?

C = 0.144013 Mbps. For the specified model T = C*x, with x = 1/sqrt(p), least squares through the origin gives C = sum(x*T) / sum(x*x), using all 70 trials. Allowing an intercept instead gives a slope of 0.163281 Mbps and intercept -0.390209 Mbps. That slope describes the affine fit on page 3; the zero-intercept estimate above is C for the writeup's proportional model.

2.3 Pearson correlation between 1/sqrt(p) and throughput

Pearson r = 0.91302 across all 70 (1/sqrt(p), throughput) pairs. It is computed as the centered cross-product divided by the product of the centered vector lengths. For comparison, the correlation using only the seven group means is 0.95959; the all-trial value is the primary result and retains the variation between repeated transfers.

2.4 Do the data corroborate Dr. Mathis's hypothesis?

The pure proportional model does not adequately describe the entire loss range. The positive correlation supports the inverse-square-root trend, but it does not establish one constant C across these experiments. For exact proportionality, T*sqrt(p) would be constant. The seven normalized means range from 0.01219 to 0.20457 Mbps, a factor of 16.79. The nonzero unconstrained intercept and trial variability also show departures from the hypothesis. The measured build's ACK-credit defect (Section 1.2) is an additional confounder; these data cannot establish the corrected implementation's behavior or refute the general Reno model.

At low loss, the 5 Mbps rate limit, queueing delay, receive window, and finite slow-start period can cap or reduce throughput. At higher loss, multiple missing packets and the fixed three-second retransmission timeout make Reno spend time waiting rather than following the steady-state sawtooth assumed by the approximation. These effects can lower throughput relative to a model based only on p and a nominal 40 ms RTT. The configured RTT is a baseline; serialization and queueing increase actual RTT during a transfer.

For the ten 10% loss trials, the number of recorded timeouts multiplied by the fixed 3 s RTO accounts for approximately 95.3% of total transfer time. This estimates the contribution of timeout intervals; it is not a direct measurement of link idleness.

Validation and reproducibility

The measured build passed 36 tests on macOS and AWS Ubuntu 24.04. The corrected build passes 38 tests on macOS, including the two new flight-growth regressions; both validations use AddressSanitizer and UndefinedBehaviorSanitizer. Commands and coverage are in tests.txt. The accompanying experiments/results directory contains all 70 timings, hashes, raw CWND traces, traffic-control settings, and regression results. source.json and remote-builds.json record matching protocol sources on both nodes. build_cp2_report.py reproduces the figures and calculations. cleanup.json records removal of the temporary AWS resources.

References

Course Project 2 writeup (October 6, 2026), Section 3.2, Figure 2, and report requirements on pages 7-8.
RFC 5681, TCP Congestion Control, Sections 3.1-3.2: https://www.rfc-editor.org/rfc/rfc5681